

Dynamic Pruning of Learned Sparse Representations using Cluster-Partitioned Inverted Indexes

Roger Canal Estrada

Universitat Politècnica de Catalunya (UPC)
Facultat d'Informàtica de Barcelona (FIB)
`roger.canal@estudiantat.upc.edu`

Abstract. Learned sparse representations preserve lexical interpretability while making large-scale retrieval harder than classical inverted indexing. This paper describes the Seed 42 implementation for SISAP 2026 Task 3: documents are clustered with a sparse K-means-style procedure, each cluster is summarized with per-dimension maximum weights, and the resulting cluster-partitioned inverted index is pruned at query time using cluster upper bounds. On the NQ SPLADE-v3 collection (2.68M documents, 30,522 dimensions), the latest sweep reaches Recall@30 = 0.9418 at 43.7 ms/query and a fastest operating point of 32.2 ms/query at Recall@30 = 0.9034.

Keywords: Sparse Retrieval · Approximate Nearest Neighbor Search · Dynamic Pruning · Learned Sparse Representations

1 Introduction

The emergence of Learned Sparse Representations (LSR), such as SPLADE [4], revitalizes inverted index-based retrieval by mapping contextualized embeddings back into the vocabulary space. This makes retrieval interpretable, but it also breaks the Zipfian assumptions that classical dynamic pruning methods, such as WAND [1], rely on for efficiency. In learned sparse vectors, many more coordinates carry meaningful mass, so score-at-a-time pruning becomes far less selective.

Recent work shows that block-wise summaries can restore much of that lost selectivity by tightening score upper bounds over semantically coherent document groups [2,3]. Following that direction, the present implementation clusters the collection globally, truncates a concept-by-cluster inverted index, and applies query-time upper-bound pruning over the retained blocks. The rest of the paper describes the implemented pipeline and reports the latest experimental trade-offs on the SISAP NQ benchmark.

2 System Architecture

The system consists of an indexing pipeline and a search engine. The indexing pipeline is subdivided into a global document clustering phase and a partitioned index construction phase.

2.1 Document Clustering Phase

The first stage applies a sparse K-means-style clustering algorithm to the entire document collection. Centroids are initialized from random documents, each document is assigned to the centroid with the highest dot product, and centroids are recomputed by summing the sparse vectors in each cluster and normalizing the result. Empty clusters are reinitialized from random documents. The goal is not to route each query to a single cluster, but to create semantically coherent blocks whose term weights can be bounded more tightly during search.

2.2 Partitioned Index

After clustering, the indexer computes one dense summary vector per cluster. For every dimension, the summary vector stores the maximum weight observed in that cluster, which later serves as an upper bound during query evaluation. The partitioned index is then built concept by concept: for each term, the implementation aggregates candidate documents per cluster, keeps the strongest clusters by accumulated term weight, and stores only the strongest documents in each retained block. The build-time truncation is controlled by nb (maximum blocks per dimension) and nd (maximum documents per block), matching Algorithm 1.

Algorithm 1 Partitioned Index Construction

Require: Clustered documents, nb (max blocks), nd (max docs)

Ensure: Partitioned Inverted Index I

```
1: for each concept  $c \in Vocabulary$  do
2:   Collect candidate documents and accumulated weights for each cluster contain-
      ing  $c$ 
3:   Sort clusters by accumulated weight for  $c$  and keep the top  $nb$ 
4:   for each retained cluster  $k$  do
5:     Sort documents in  $k$  by weight for  $c$  and keep the top  $nd$ 
6:     Create InvertedBlock( $k, doc.ids, weights$ )
7:     Append the block to  $I[c]$ 
8:   end for
9: end for
10: return  $I$ 
```

2.3 Search Engine

The search engine processes non-zero query coordinates in descending weight order, computes a per-cluster upper bound from the summary vectors, and scans the blocks stored for each concept. If a block’s upper bound falls below the current heap threshold relaxed by the `heap_factor` hyperparameter, the whole block is skipped. Otherwise, the engine evaluates exact dot products only for previously unseen documents in that block and maintains a min-heap of the top- k results. Search stops once the hard document budget md is reached.

Algorithm 2 Search Engine

Require: Query q , Index I , Summaries, `heap_factor`, md , $Results_K$

Ensure: Top- $Results_K$ documents

```

1:  $SortedTerms \leftarrow$  Sort non-zero terms in  $q$  by weight descending
2:  $Heap \leftarrow$  Initialize Min-Heap of size  $Results\_K$ 
3:  $ClusterUB \leftarrow$  Compute upper bounds for all clusters using  $q$  and Summaries
4:  $DocsExamined \leftarrow 0$ 
5: for each term  $t \in SortedTerms$  do
6:   for each InvertedBlock  $B \in I[t]$  do
7:     if  $ClusterUB[B.cluster\_id] < Heap.min() \times \text{heap\_factor}$  then
8:       continue ▷ Aggressively skip entire block
9:     end if
10:    for each document  $d \in B.doc\_ids$  do
11:      if  $DocsExamined \geq md$  then
12:        return  $Heap$ 
13:      end if
14:       $score \leftarrow \text{ExactDotProduct}(q, d)$ 
15:      Update  $Heap$  with  $(score, d)$ 
16:       $DocsExamined \leftarrow DocsExamined + 1$ 
17:    end for
18:  end for
19: end for
20: return  $Heap$ 

```

3 Experimental Evaluation

I evaluate the system on the 2.68 million document Natural Questions (NQ) dataset. The embeddings are SPLADE-v3 vectors with 30,522 dimensions. Retrieval quality is measured with Recall@30 against the exact k -nearest neighbors provided by the dataset.

3.1 Hyperparameter Configuration

To properly frame the evaluation, an execution is defined by the hyperparameters introduced in the previous section:

Table 1. Evidence for selecting $K = 2000$ and $itr = 3$.

Setting	Recall@30	ms/query	Clust. time (s)	Total time (s)
$K = 2000, itr = 3$ (final)	0.9034	32.24	839	897.6
$K = 3000, itr = 4$ (exploratory)	0.8412	33.68	1621	1660.4
$K = 4000, itr = 4$ (exploratory)	0.8266	38.93	2313	2357.4
$itr = 3$ avg (fixed sweep, $md = 70000$)	0.9220	38.80	–	–
$itr = 5$ avg (fixed sweep, $md = 70000$)	0.9209	38.89	–	–

- **K** : The number of global clusters.
- **nb & nd** : The index breadth (max blocks) and depth (max docs per block).
- **md** : The hard cap on total documents examined during search.
- **$heap_factor$** : The relaxation multiplier governing dynamic block skipping.

The latest sweep fixes $K = 2000$ and explores the parameter ranges shown in Table 3. The task target is $\text{Recall@30} \geq 0.90$, so the most relevant operating points are the fastest configuration that meets that threshold and the best-recall configuration in the same sweep.

3.2 Hyperparameter Selection Process

Before selecting final operating points, I evaluate each hyperparameter family with controlled slices from the experiment logs.

Selecting K and itr for clustering. Table 1 summarizes the evidence used to select $K = 2000$ and $itr = 3$ as the default preprocessing setup. The itr choice comes from the fixed-engine sweep averages (same search budgets), where $itr = 3$ is consistently better in recall and similar in latency. The K choice is based on exploratory scaling runs: moving from $K = 2000$ to $K \in \{3000, 4000\}$ substantially increases preprocessing time without improving recall in comparable settings.

Relation between nb and nd . Figure 1 shows the $itr = 3$, $md = 60000$ slice of the fixed-engine sweep. The strongest pattern is that larger nd tends to reduce recall for the same nb (e.g., for $nb = 300$: 0.915 at $nd = 100$ vs 0.896 at $nd = 150$), while larger nb beyond 300 does not improve recall in this budget regime. This indicates that, under a hard document budget, allocating capacity to breadth-only growth does not guarantee better candidate quality.

Effect of $heap_factor$. Table 2 reports two regimes. In early exploratory runs, a high $heap_factor$ (0.8) causes over-pruning (very few blocks entered), which collapses recall. In the final sweep, low values (0.10–0.15) keep pruning permissive enough to satisfy $\text{Recall@30} \geq 0.90$.

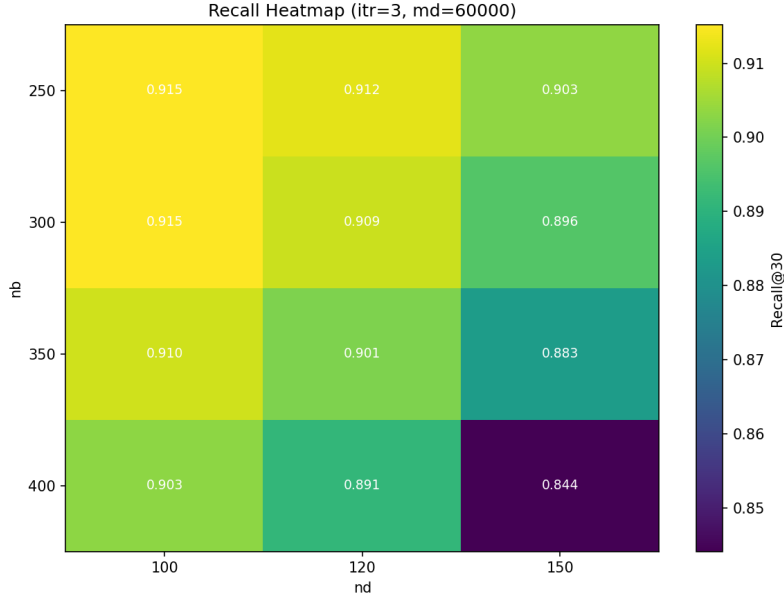


Fig. 1. Recall@30 sensitivity to nb and nd for $itr = 3$, $md = 60000$.

Table 2. Observed effect of `heap_factor` on pruning aggressiveness and recall.

Configuration	hf	Recall@30	Docs examined	Blocks ent
Exploratory ($K = 3000$, $itr = 4$, $nb = 300$, $nd = 150$, $md = 60000$)	0.80	0.2237	308	2.9
Same exploratory config	0.10	0.8412	57021	704.0
Final operating point ($K = 2000$, $itr = 3$, $nb = 250$, $nd = 150$, $md = 60000$)	0.10	0.9034	60000	945.3

3.3 Current Results

Table 3 summarizes the latest operating points. The fastest configuration that satisfies the task target reaches Recall@30 = 0.9034 at 32.23 ms/query, while the highest recall in the sweep reaches 0.9418 at 43.73 ms/query. In both cases, the average number of examined documents equals the configured budget md , which indicates that the hard document cap is the dominant stopping condition.

3.4 Incoming Results

The main trend is consistent across the sweep: larger values of md improve recall more reliably than small adjustments to `heap_factor`, but they also raise latency. Increasing nb and reducing nd helps the system reach the Recall@30 target faster than the earlier ultra-aggressive pruning settings, yet the current implementation still spends most of its budget on exact scoring rather than on block skipping.

Table 3. Representative operating points from the latest fixed-engine sweep on NQ ($K = 2000$).

Operating point	<i>itr</i>	<i>nb</i>	<i>nd</i>	<i>md</i>	<i>hf</i>	Recall@30	ms/query
Fastest ≥ 0.90	3	250	150	60000	0.10	0.9034	32.23
Best recall	3	300	100	80000	0.10	0.9418	43.73
Balanced	3	300	120	80000	0.10	0.9409	43.45

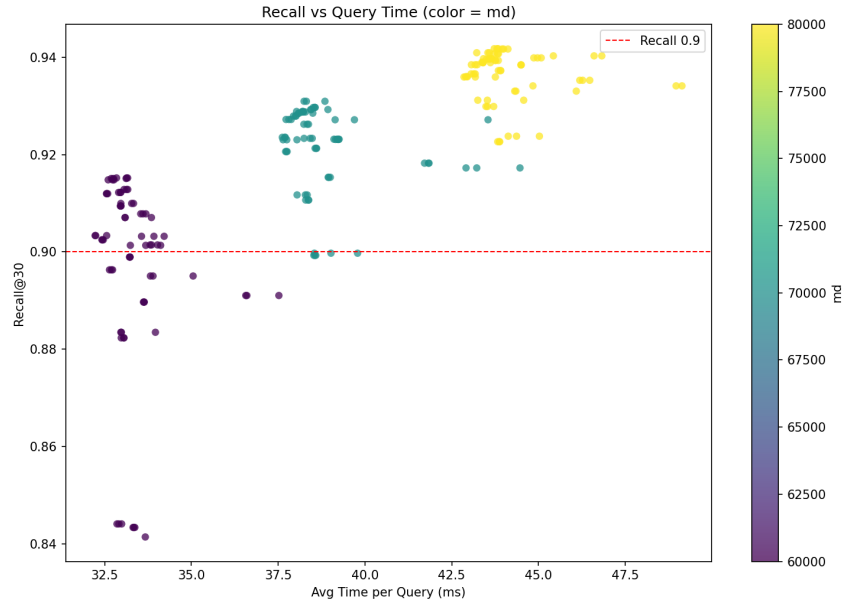


Fig. 2. Recall/latency trade-off across the latest fixed-engine sweep. Colors encode the document budget md and the dashed line marks the task target Recall@30 = 0.90.

4 Conclusion and Future Work

I present an approximate retrieval system for learned sparse vectors that combines global clustering, per-cluster summary bounds, and a truncated partitioned inverted index. The latest experiments show that the implementation can satisfy the SISAP target of Recall@30 ≥ 0.90 , with a fastest operating point at 32.2 ms/query and a best-recall operating point at 43.7 ms/query. The main remaining limitation is that the search phase often reaches the document budget md , so future improvements should focus on tighter candidate generation or a more selective block-ranking strategy rather than more aggressive pruning alone.

References

1. Broder, A. Z., Carmel, D., Herscovici, M., Soffer, A., & Zien, J. (2003). Efficient query evaluation using a two-level retrieval process. In *Proceedings of the 12th International Conference on Information and Knowledge Management (CIKM)*.
2. Bruch, S., Nardini, F. M., Rulli, C., & Venturini, R. (2024). Efficient Inverted Indexes for Approximate Retrieval over Learned Sparse Representations. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.
3. Bruch, S., Nardini, F. M., Rulli, C., & Venturini, R. (2024). Pairing Clustered Inverted Indexes with κ -NN Graphs for Fast Approximate Retrieval over Learned Sparse Representations. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*.
4. Formal, T., Piwowarski, B., & Clinchant, S. (2021). SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*.